

Accelerating Learning Via Metaexamples

Kevin Lin

Claude Code*

February 2026

Abstract

We investigate whether natural language descriptions of rules—which we call *metaexamples*—can accelerate a language model’s learning of those rules from examples. Using an artificial grammar with nonsense tokens, we fine-tune Pythia 1.4B on mixtures of grammar examples and metaexamples via continued pretraining. We find that adding just 1% metaexamples to the training mix significantly improves grammar validity at every level of pretraining maturity, while 10% metaexamples hurts and 100% metaexamples (descriptions alone, no examples) produces 0% validity. These results suggest that pretrained LLMs learn a bidirectional bridge between examples and their explanations, and that a small amount of explicit rule description can help models extract structure from examples more effectively.

1 Introduction

It has been demonstrated by Treutlein et al. [1] that LLMs can perform *inductive out-of-context reasoning* (OOCR)—inferring latent structure from disparate training examples and verbalizing it at test time. In one of their experiments, a model is fine-tuned on documents of the form

```
from coins import CoinA; print(CoinA.flip()) → Heads or Tails
```

repeated across hundreds of examples with varying outcomes. The model never sees an explicit statement of the coin’s bias. Yet when asked “What is the probability that CoinA lands heads?”, it can give a reasonable estimate. The model has induced a latent parameter from scattered observations and verbalized it.

This is a striking and somewhat mysterious result. The model is not merely memorizing and recombining training data like a “stochastic parrot” [3]—it is also performing induction over the training data, extracting structure that was never explicitly stated. How does this happen? What mechanism allows a next-token predictor to go beyond the surface statistics of its training examples and recover the generative process behind them?

A clue is offered by “Textbooks Are All You Need” [2], which showed that training on textbook data dramatically improves sample efficiency—their 1.3B parameter model matched much larger models on code generation. We observe that textbooks are notable in that they contain two types of information side-by-side:

1. *Examples* of phenomena—worked problems, code snippets, instances (like the results of coinflips)

*Code available at <https://github.com/tractatus1889/hyperdata>

2. *Metaexamples*—descriptions of the underlying structure or rules that govern those phenomena—definitions, explanations, theorems (like a coin’s bias)

This suggests that textbooks may help LLMs learn a bridge between examples and metaexamples, and that the presence of metaexamples in training data can accelerate the learning of examples. In this view, we may rephrase the result of [1] as: If a model has learned to bridge examples and metaexamples during pretraining, then fine-tuning on examples alone will make the model try to infer the underlying structure which would be explained by metaexamples.

In this paper, we explore the *reverse* direction to [1]: Can metaexamples help a model become better at producing valid examples? If there is indeed a learned bridge between examples and their explanations, then the bridge should be traversable in both directions.

To investigate this question, we create an artificial grammar, and we generate

1. *Examples* from this grammar
2. *Metaexamples* consisting of natural language descriptions of aspects of the grammar

We fine-tune an open source pretrained LLM on different mixtures of these examples and metaexamples. Our main result is that the presence of metaexamples can in fact accelerate the learning of examples. As shown in Figure 1, adding just 1% metaexamples to the training mix improves grammar validity at every level of pretraining maturity, with the strongest effect at the earliest checkpoint.

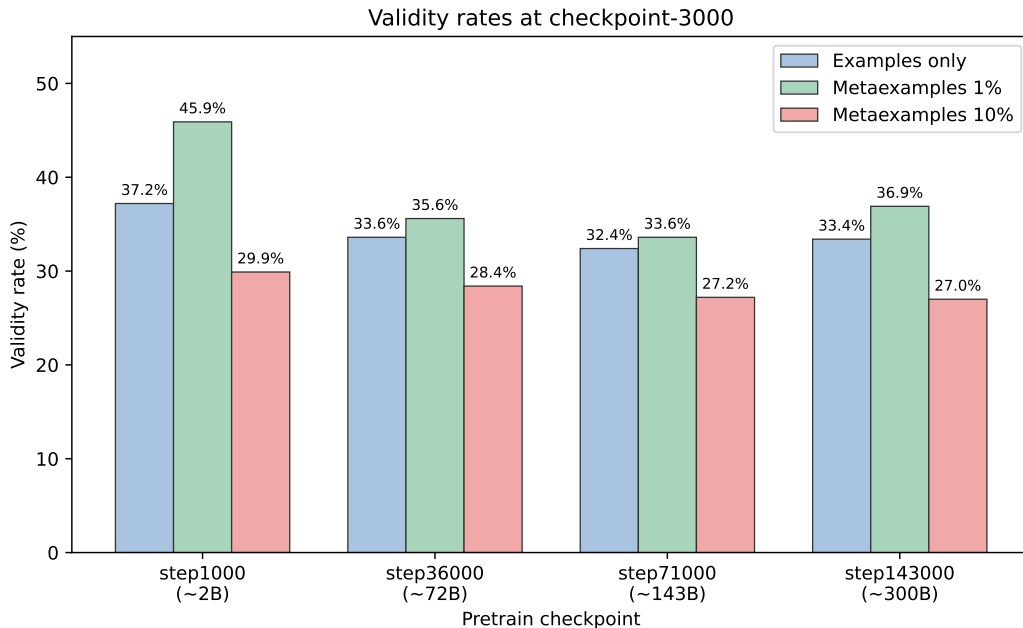


Figure 1: Validity rates at checkpoint-3000. Metaexamples 1% (green) outperforms examples-only (blue) at every pretrain checkpoint, with the largest gap at step1000. Metaexamples 10% (red) consistently underperforms.

2 Grammar

We define a fictional grammar that we call Tivari3 consisting of nonsense tokens and arbitrary rules. We do this to ensure that there is no chance for the pretrained model to have any priors on the grammar. This way, the model’s learning of the grammar must be done “from scratch”.

The grammar is defined as follows:

1. Expressions are wrapped in `<tivari3>` / `</tivari3>` tags.
2. The first token must be FEP.
3. The last token must be GOR.
4. Content between FEP and GOR must be made from the tokens NUL, TAS, WEJ, KOB, and no other tokens.
5. Content between FEP and GOR must be a **palindrome**.
6. TAS and WEJ must each appear an **even** number of times.

Example valid expressions in Tivari3:

- `<tivari3> FEP GOR </tivari3>`
- `<tivari3> FEP WEJ WEJ GOR </tivari3>`
- `<tivari3> FEP NUL NUL GOR </tivari3>`
- `<tivari3> FEP TAS KOB TAS GOR </tivari3>`

Example invalid expressions in Tivari3:

- `<tivari3> FEP NUL TAS GOR </tivari3>` (not palindrome)
- `<tivari3> FEP WEJ GOR </tivari3>` (WEJ appears once)
- `<tivari3> FEP WEJ TAS WEJ GOR </tivari3>` (palindrome but TAS appears once)

In our experiment, example training documents each contain a single valid Tivari3 expression (like those above), while metaexample documents each contain a single natural language sentence about the grammar. The full set of metaexamples (which may be repeated in fine-tuning data):

- A Tivari3 expression is enclosed in `<tivari3>` and `</tivari3>` tags.
- In Tivari3, every valid expression starts with FEP and ends with GOR.
- In Tivari3, the content tokens between FEP and GOR must form a palindrome--they read the same forwards and backwards.
- The allowed content tokens in Tivari3 are NUL, TAS, WEJ, and KOB.
- In Tivari3, the token TAS must appear an even number of times (0, 2, 4, and so on).
- In Tivari3, the token WEJ must appear an even number of times (0, 2, 4, and so on).
- FEP NUL TAS GOR is invalid Tivari3 because NUL TAS is not a palindrome.
- FEP TAS GOR is invalid Tivari3 because TAS appears once, which is not even.
- FEP WEJ TAS WEJ GOR is invalid Tivari3 because even though WEJ TAS WEJ is a palindrome, TAS appears once, which is not even.

3 Experiment Setup

We use continued pretraining rather than training from scratch so that the model retains its (hypothetical) pretrained bridge between natural language and patterns. The 10% synthetic / 90%

C4 mix ensures the model does not catastrophically forget its pretraining distribution while still receiving enough synthetic data to learn the grammar.

- **Pretrain model:** EleutherAI’s Pythia 1.4B at 4 pretraining checkpoints:
 - step1000 (~ 2 B tokens)
 - step36000 (~ 72 B tokens)
 - step71000 (~ 143 B tokens)
 - step143000 (~ 300 B tokens, fully pretrained)
- **Finetuning:** 3000 continued pretraining steps, $LR=1 \times 10^{-5}$, batch size=4, gradient accumulation steps=8, warmup steps=1000
- **4 types of synthetic data:**
 - 100% examples
 - 99% examples + 1% metaexamples
 - 90% examples + 10% metaexamples
 - 0% examples + 100% metaexamples
- **Eval:** Check validity of completions for 2 prompts (`<tiviari3>`, `<tiviari3> FEP`), 10,000 samples per prompt, temperature=1.0

4 Results

Table 1 shows validity rates at checkpoint-3000 across all conditions.

Pretrain checkpoint	Examples only	1% metaex.	10% metaex.	100% metaex.
step1000 (~ 2 B)	37.2%	45.9%	29.9%	0%
step36000 (~ 72 B)	33.6%	35.6%	28.4%	0%
step71000 (~ 143 B)	32.4%	33.6%	27.2%	0%
step143000 (~ 300 B)	33.4%	36.9%	27.0%	0%

Table 1: Validity rates at checkpoint-3000 across pretraining maturity levels and metaexample ratios. All differences between conditions are statistically significant (two-proportion z-test, $n = 20,000$ per condition, $p < 0.01$).

4.1 1% metaexamples improves grammar learning

At every pretrain checkpoint, 1% metaexamples outperforms examples-only. The effect is present at all pretraining maturity levels and is strongest at step1000.

Interpretation: This validates our hypothesis that the bridge between examples and metaexamples is bidirectional.

4.2 10% metaexamples hurts

At every pretrain checkpoint, 10% metaexamples underperforms examples-only.

Interpretation: The metaexamples may be repeated too many times and learning capacity is being spent on memorizing the exact phrasing of the metaexamples rather than the information that they contain.

4.3 Less pretraining learns the grammar better

Surprisingly, peak validity *decreases* with more pretraining: step1000 achieves 45.9% vs 36.9% at step143000 (both with 1% metaexamples).

Interpretation: More pretrain steps translate to stronger priors that resist learning the nonsense grammar. This result also suggests that the bridge between examples and metaexamples already exists early on, at step1000—and this may reflect a basic capability (connecting English descriptions to patterns) rather than something deeply learned from textbook-like data.

4.4 Explanations alone are not sufficient

100% metaexamples produces 0% validity at every pretrain checkpoint.

Interpretation: The model cannot learn to generate valid strings from descriptions alone—it needs examples. We need both sides of the bridge to be present to learn it. This differs from the result in [1], but note that their experiments involve well-known concepts like probability, whereas we use a completely novel grammar.

5 Error Analysis

We sampled invalid generations for analysis. All invalid outputs have correct structure (FEP ... GOR with valid tokens)—the model learns the format. Errors fall into two categories:

- **Not a palindrome:** The dominant failure. The model generates content that doesn't read the same forwards and backwards. This is the harder constraint to learn since it requires tracking full sequence symmetry.
- **Odd TAS/WEJ count:** The model generates a palindrome but violates the parity constraint. Less common.

5.1 Sample errors by condition (step143000)

Examples only—mostly palindrome failures:

- FEP TAS WEJ WEJ TAS TAS TAS WEJ WEJ GOR (not palindrome)
- FEP NUL KOB NUL NUL GOR (not palindrome)

Metaexamples 1%—more parity-only errors, which may suggest the model learned the palindrome constraint better:

- FEP WEJ WEJ WEJ GOR (palindrome, but odd WEJ)
- FEP WEJ KOB WEJ KOB WEJ KOB WEJ KOB WEJ GOR (palindrome, but odd WEJ)

Metaexamples 10%—mixed errors:

- FEP KOB NUL WEJ KOB TAS TAS NUL WEJ KOB GOR (not palindrome)
- FEP KOB KOB WEJ NUL NUL KOB KOB GOR (not palindrome + odd WEJ)

6 Conclusion

Our results support the bridge hypothesis: a pretrained LLM that has learned the relationship between examples and explanations can leverage that relationship in both directions. It was shown

in [1] that examples alone can help models infer latent rules. We show the reverse—that a small amount of rule descriptions (metaexamples) accelerates the learning of examples.

The effect is robust across pretraining maturity levels but strongest when the model has the fewest priors (step1000), suggesting the bridge is easier to exploit when competing knowledge is weaker. That the effect exists so early in pretraining also suggests that the relation between examples and metaexamples—or we might say information and metainformation—may be a very basic, fundamental property of language.

We also find that a metaexample fraction of 1% works better than 10%, possibly because learning capacity is being spent memorizing the phrasing of metaexamples rather than learning the structural information they contain.

Several limitations apply. We test a single artificial grammar on one model family (Pythia 1.4B). Our metaexamples are generated descriptions of a single known grammar; in realistic settings, the quality and accuracy of explanations would vary. Future work could test whether the effect scales to larger models, more complex grammars, and natural (rather than artificial) domains. It would also be valuable to vary the pretraining data composition—for example, we might test whether models trained on more textbook-like data have a stronger bridge and benefit even more from metaexamples.

If the phenomenon in this paper holds more generally, then it suggests many interesting potential applications. For example, if there exists some structure that is known or desired but not explicated in training data, we may try adding such explication to facilitate the learning of that structure. If we want a code model to follow a particular style guide, we might add natural language descriptions of the style rules alongside code examples, rather than relying on the model to infer the rules from examples alone.

References

- [1] Treutlein, J., Choi, D., Betley, J., Marks, S., Anil, C., Grosse, R., & Evans, O. (2024). Connecting the Dots: LLMs Can Infer and Verbalize Latent Structure from Disparate Training Data. *arXiv:2406.14546*. <https://arxiv.org/abs/2406.14546>.
- [2] Gunasekar, S., Zhang, Y., Anber, J., Hejazinia, H., Bubeck, S., et al. (2023). Textbooks Are All You Need. *arXiv:2306.11644*. <https://arxiv.org/abs/2306.11644>.
- [3] Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? *Proceedings of FAccT 2021*. <https://doi.org/10.1145/3442188.3445922>.